

SERTAO

System for Estimation of cosmological paRameTers
and Analysis of Observables
Manual do Usuário

Rafael C. Nunes

Contents

1	Introdução	3
2	Requisitos e Instalação	3
2.1	Requisitos	3
2.2	Instalação	3
2.3	Estrutura de diretórios	4
3	Como Executar uma Análise	5
3.1	O ponto de entrada: <code>run.py</code>	5
3.2	O que aparece na tela durante a execução	5
3.3	Retomada automática (<i>checkpoint</i>)	7
3.4	Arquivos de saída	7
4	Datasets Disponíveis	7
5	Modelos Disponíveis em <code>theory_models/</code>	8
5.1	<code>LCDM.py</code> — Λ CDM	9
5.2	<code>LCDM_neutrinos.py</code> — Λ CDM + neutrinos massivos	9
5.3	<code>wCDM.py</code> — Energia escura com w constante	9
5.4	<code>w0wacDM.py</code> — Parametrização CPL (energia escura dinâmica)	10
6	Como Criar um Novo Modelo	10
6.1	Exemplo: modelo w CDM simples	11
6.2	Diretrizes para a função <code>H_model</code>	11

7	Componentes Internos	12
7.1	<code>engine.py</code> — Motor de likelihood	12
7.2	<code>cosmology.py</code> — Distâncias e crescimento	12
7.3	<code>neutrinos.py</code> — Neutrinos massivos	13
7.4	<code>MCMC.py</code> — Nested sampling	13
8	Função de Crescimento: Quando Modificar?	14
8.1	Extensão para gravidade modificada: $\mu(a)$	14
9	Módulo de Campo Escalar (<code>scalar_field.py</code>)	15
10	Cosmologia Híbrida com CLASS (<code>hybrid_cosmology.py</code>)	16
11	Testes Automatizados	16
12	Eficiência Computacional	17
12.1	Tempos esperados de execução	17
12.2	Recomendações	17
13	Resumo: O que o Usuário Precisa Fazer	18

1 Introdução

O **SERTAO** é um *framework* em Python para estimativa estatística de parâmetros cosmológicos a partir de dados observacionais. O código foi projetado para ser modular e de uso direto: o usuário define apenas o modelo cosmológico de interesse e o SERTAO cuida de todo o resto, desde o cálculo das *likelihoods* até a geração e o salvamento das cadeias.

A ideia central é clara: o usuário interage com **um único arquivo** de configuração, enquanto toda a infraestrutura interna — engine de *likelihood*, amostragem via *nested sampling*, salvamento de resultados — opera de forma completamente automática e transparente.

Em termos práticos, o usuário precisa apenas:

- definir o modelo cosmológico por meio de uma função $H(z, p)$;
- escolher os conjuntos de dados observacionais;
- especificar os *priors* dos parâmetros;
- executar a análise com um único comando.

Nota

Nenhum conhecimento prévio do código interno é necessário para usar o SERTAO. A leitura das seções 2 e 3 é suficiente para iniciar uma análise.

2 Requisitos e Instalação

2.1 Requisitos

O SERTAO requer apenas uma instalação padrão de Python e um conjunto reduzido de bibliotecas científicas amplamente disponíveis:

- **Python** ≥ 3.9 (versões 3.10, 3.11, 3.12 são suportadas)

As dependências externas são instaladas via `pip`:

```
1 pip install numpy scipy matplotlib pandas dynesty mpi4py
```

2.2 Instalação

No diretório raiz do projeto, execute:

```
1 pip install -e .
```

A opção `-e` (modo editável) é recomendada durante o uso: qualquer alteração nos arquivos é refletida imediatamente, sem necessidade de reinstalar.

2.3 Estrutura de diretórios

Após a instalação, o projeto tem a seguinte organização:

```
1 SERTA0/
2 |
3 |-- run.py                # Ponto de entrada: executa qualquer
   modelo
4 |-- test_likelihoods.py    # Suite de testes (21 testes automatizados
   )
5 |-- plot_chains_example.py # Exemplo de visualizacao de resultados
6 |
7 |-- theory_models/        # Modelos cosmologicos prontos para uso
8 |   |-- LCDM.py           # Lambda-CDM padrao
9 |   |-- LCDM_neutrinos.py  # Lambda-CDM + neutrinos massivos
10 |   |-- wCDM.py           # Energia escura w = constante
11 |   |-- wOwaCDM.py        # Parametrizacao CPL (w0, wa)
12 |   |-- scalar_field_run.py # Quintessencia (campo escalar)
13 |
14 |-- SERTA0/              # Biblioteca interna (nao modificar)
15 |   |-- engine.py         # Motor de likelihood
16 |   |-- cosmology.py      # Distancias e crescimento de estruturas
17 |   |-- neutrinos.py      # Densidade de neutrinos (WMAP-7)
18 |   |-- MCMC.py           # Nested sampling e saida de resultados
19 |   |-- scalar_field.py   # Dinamica de campo escalar
20 |   '-- hybrid_cosmology.py # Interface com CLASS
21 |
22 |-- likelihoods/         # Funcoes de likelihood (nao modificar)
23 |   |-- BAO_desi.py       # BAO DESI DR2
24 |   |-- BAO_2D.py         # BAO angular bidimensional
25 |   |-- PP.py             # Supernovas Pantheon+
26 |   |-- PP_SHOES.py       # Pantheon+ calibrado pelo SHOES
27 |   |-- Union3.py         # Supernovas Union 3.0
28 |   |-- BBN.py            # Nucleossintese primordial
29 |   |-- CMBTheta.py       # CMB comprimida (R, la, omega_b h^2)
30 |   |-- CC.py             # Cronometros cosmiocos
31 |   |-- rsd_likelihood.py  # Crescimento de estruturas (f*sigma8)
32 |   '-- f_likelihood.py    # Taxa de crescimento f(z)
33 |
34 |-- data/                # Dados observacionais (nao modificar)
35 '-- chains/              # Cadeias geradas (criado automaticamente)
```

3 Como Executar uma Análise

3.1 O ponto de entrada: run.py

Todo o SERTA0 é controlado por um único comando:

```
1 python run.py --model NOME_DO_MODELO
```

O argumento `-model` indica qual arquivo dentro de `theory_models/` será usado. Por exemplo:

```
1 # LCDM padrao
2 python run.py --model LCDM
3
4 # LCDM com neutrinos massivos
5 python run.py --model LCDM_neutrinos
6
7 # Modelo CPL (energia escura dinamica)
8 python run.py --model w0waCDM
```

Para rodar com MPI e gerar múltiplas cadeias independentes (recomendado para publicações):

```
1 mpirun -np 4 python run.py --model LCDM
```

Opções adicionais disponíveis:

```
1 python run.py --model LCDM --nlive 500      # mais pontos vivos (maior
    precisao)
2 python run.py --model LCDM --no-resume      # começar do zero (ignorar
    checkpoint)
3 python run.py --model LCDM --output meus_resultados/
```

3.2 O que aparece na tela durante a execução

Ao iniciar, o SERTA0 imprime um cabeçalho informativo. Em seguida, atualiza uma tabela a cada 30 segundos mostrando o estado atual da amostragem:

```
1 =====
2 SERTA0 . Cosmological parameter estimation
3 =====
4 Analysis      : LCDM
5 Parameters    : H0, Omega_cdm, Omega_b
6 Datasets      : Pantheon+, BAO_DESI, BBN
7 Method        : Nested Sampling (dynesty)
8 Live pts      : 300 | stop DlogZ < 0.1
9 Processes     : 1
10 Checkpoint    : fresh run
```

```

11  Output      : chains/
12  =====
13  Rows every 30 s | chi2_eff = -2 ln L | DlogZ stops at 0.1
14
15  -----
16  iter          H0   Omega_cdm  Omega_b   chi2_eff  logZ   DlogZ   calls
17  -----
18  312           67.41    0.2638   0.0493    23.87 -12.44  0.821   6,240
19  512           67.89    0.2612   0.0491    22.14 -12.41  0.621   11,264
20  1240          67.36    0.2642   0.0493    21.98 -12.39  0.104   27,280
21  -----
22  =====
23  Run complete v
24  Wall time   : 0:04:12
25  log Z       : -12.3847 +- 0.0523
26  Iterations  : 1,240
27  Calls       : 27,280
28
29  Best-fit parameters (max log L point):
30  H0           = 67.361234
31  Omega_cdm     = 0.264198
32  Omega_b       = 0.049312
33  chi2_eff      = 21.9841
34
35  Chains       : chains/LCDM_1.txt
36  Evidence     : chains/LCDM_logZ.txt
37  =====

```

Listing 1: Exemplo de saída no terminal durante uma análise LCDM

O significado de cada coluna da tabela:

- **iter**: número de pontos mortos já coletados pelo sampler;
- **params**: valores dos parâmetros do ponto sendo descartado — convergem em torno do melhor ajuste ao longo do tempo;
- **chi2_eff**: $-2 \ln \mathcal{L}$ do ponto atual (diminui conforme a análise avança);
- **logZ**: evidência Bayesiana acumulada;
- **$\Delta \log Z$** : evidência restante — a análise para quando este valor cai abaixo de **dlogz** (padrão: 0.1);
- **calls**: total de avaliações de likelihood realizadas.

3.3 Retomada automática (*checkpoint*)

Se a análise for interrompida (queda de energia, tempo limite de cluster, etc.), basta rodar o mesmo comando novamente. O SERTAIO detecta automaticamente o *checkpoint* salvo e retoma de onde parou:

```
1 python run.py --model LCDM      # retoma automaticamente se existir
    checkpoint
```

Para forçar uma análise do zero, eliminando o checkpoint anterior:

```
1 python run.py --model LCDM --no-resume
```

3.4 Arquivos de saída

Ao final da execução, os seguintes arquivos são gerados no diretório `chains/` (ou no definido por `-output`):

- `LCDM_1.txt`, `LCDM_2.txt`, ...: cadeias no formato padrão do `GetDist` (colunas: peso, $-\ln \mathcal{L}$, parâmetros);
- `LCDM.paramnames`: nomes e símbolos $\text{\LaTeX}$ dos parâmetros;
- `LCDM.ranges`: intervalos dos priors;
- `LCDM_logZ.txt`: evidência Bayesiana $\ln Z \pm \sigma_Z$.

Esses arquivos são compatíveis com o `GetDist` e podem ser usados diretamente para gerar contornos de confiança.

4 Datasets Disponíveis

O SERTAIO implementa as seguintes likelihoods observacionais:

Nome	Arquivo	Descrição
BAO_DESI	BAO_desi.py	BAO do DESI DR2 (7 bins, $0.3 < z < 2.3$)
BAO_2D	BAO_2D.py	BAO angular bidimensional
Pantheon+	PP.py	1701 supernovas Ia ($0.01 < z < 2.3$)
Pantheon+SHOES	PP_SHOES.py	Pantheon+ calibrado com âncora Cepheids
Union3	Union3.py	Union 3.0 (2087 supernovas Ia)
BBN	BBN.py	Prior gaussiano em $\omega_b h^2$ (Planck 2018)
CMB θ^*	CMBTheta.py	CMB comprimida: $R, \ell_a, \omega_b h^2$
CC	CC.py	Cronômetros cósmicos $H(z)$
RSD	rsd_likelihood.py	Crescimento de estruturas $f\sigma_8(z)$
f	f_likelihood.py	Taxa de crescimento $f(z)$

Atenção

Pantheon+ e **Union3** não podem ser usados simultaneamente — medem o mesmo observável (distância de luminosidade) com catálogos diferentes. O SERTAO detecta essa combinação e retorna um erro explicativo antes de iniciar a análise.

Parâmetros de incômodo adicionados automaticamente. Alguns datasets requerem parâmetros de incômodo livres. O arquivo de modelo cuida disso automaticamente:

- Pantheon+ ou Pantheon+SHOES: adiciona $M_B \in (-21, -18)$;
- Union3: adiciona $M_{\text{cal}} \in (-20, -17)$;
- RSD ou f: adiciona $\sigma_8 \in (0.5, 1.0)$.

5 Modelos Disponíveis em theory_models/

Cada arquivo em `theory_models/` é autossuficiente: define os datasets, os priors e a função $H(z)$ para um modelo cosmológico específico. O usuário configura apenas o arquivo do modelo desejado.

5.1 LCDM.py — Λ CDM

Modelo padrão: universo plano com constante cosmológica Λ e radiação de fótons. Parâmetros livres: H_0 , Ω_{cdm} , Ω_b .

$$H^2(z) = H_0^2 [\Omega_m(1+z)^3 + \Omega_r(1+z)^4 + \Omega_\Lambda], \quad (1)$$

onde $\Omega_\Lambda = 1 - \Omega_m - \Omega_r$ (universo plano) e $\Omega_r = \omega_\gamma/h^2$, com $\omega_\gamma = 2.469 \times 10^{-5}$ (Fixsen 2009).

5.2 LCDM_neutrinos.py — Λ CDM + neutrinos massivos

Extensão do Λ CDM que inclui neutrinos massivos de forma rigorosa. A densidade de neutrinos é calculada pelo formalismo WMAP-7 (Komatsu et al. 2011, Apêndice C), que rastreia a transição relativístico $\rightarrow$ não-relativístico espécie a espécie por meio da integral de Fermi-Dirac:

$$\Omega_\nu(z) = 0.2271 \frac{N_{\text{eff}}}{N_{\text{esp}}} \Omega_\gamma(z) \sum_i f\left(\frac{m_i}{T_\nu(z)}\right), \quad (2)$$

onde $T_\nu(z) = T_{\nu,0}(1+z)$, $T_{\nu,0} = (4/11)^{1/3} T_{\text{CMB}} = 1.945 \text{ K}$, e a função de Fermi-Dirac é aproximada por

$$f(y) \approx [1 + (0.3173 y)^{1.83}]^{1/1.83}, \quad y = \frac{m_i}{T_\nu(z)}. \quad (3)$$

Esta aproximação tem precisão melhor que 0.35% para todo $y \in [0, \infty)$.

O arquivo oferece quatro modos configuráveis:

SAMPLE_NU_MASS	SAMPLE_NEFF	Modelo	Uso típico
False	False	Λ CDM + ν fixos	Análise padrão
True	False	Λ CDM + Σm_ν livre	Limite em massa
False	True	Λ CDM + N_{eff} livre	Radiação escura
True	True	Λ CDM + $\Sigma m_\nu + N_{\text{eff}}$	Análise geral

Configurações suportadas para a hierarquia de massas: **standard** (3 espécies degeneradas), **1ncdm**, **2ncdm**, **hierarchy_normal**, **hierarchy_inverted**.

5.3 wCDM.py — Energia escura com w constante

Generalização do Λ CDM em que a equação de estado da energia escura é um parâmetro livre constante w_0 :

$$H^2(z) = H_0^2 [\Omega_m(1+z)^3 + \Omega_r(1+z)^4 + \Omega_{\text{DE0}}(1+z)^{3(1+w_0)}]. \quad (4)$$

A expressão $(1+z)^{3(1+w_0)}$ é exata para $w = \text{const}$ e não requer integração numérica. O caso $w_0 = -1$ recupera o ΛCDM .

Prior padrão: $w_0 \in (-3, 1)$.

5.4 w0waCDM.py — Parametrização CPL (energia escura dinâmica)

Implementa a parametrização de Chevallier–Polarski–Linder (CPL):

$$w(a) = w_0 + w_a(1 - a) = w_0 + w_a \frac{z}{1+z}. \quad (5)$$

A densidade de energia escura evolui segundo:

$$\frac{\rho_{\text{DE}}(z)}{\rho_{\text{DE}}(0)} = (1+z)^{3(1+w_0+w_a)} \exp\left(-\frac{3w_a z}{1+z}\right). \quad (6)$$

Esta fórmula fechada é obtida integrando analiticamente a equação de continuidade e é numericamente estável para todo $z \geq 0$. Não há integração numérica de ρ_{DE} .

O parâmetro de Hubble é então:

$$H^2(z) = H_0^2 \left[\Omega_m(1+z)^3 + \Omega_r(1+z)^4 + \Omega_{\text{DE}0} (1+z)^{3(1+w_0+w_a)} e^{-3w_a z/(1+z)} \right]. \quad (7)$$

Limites:

- $w_a = 0$: recupera o modelo $w\text{CDM}$;
- $w_0 = -1, w_a = 0$: recupera o ΛCDM .

Priors padrão (compatíveis com DESI DR2): $w_0 \in (-3, 1)$, $w_a \in (-3, 2)$.

6 Como Criar um Novo Modelo

Criar um novo modelo no SERTA0 é simples: basta criar um arquivo `.py` em `theory_models/` com três elementos obrigatórios.

1. **DATASETS**: lista de datasets a usar.
2. **PRIORS**: dicionário com os intervalos dos parâmetros.
3. **H_model(z, p)**: função que retorna $H(z)$ em km/s/Mpc, dado o vetor de parâmetros `p`.

6.1 Exemplo: modelo w CDM simples

```
1 import numpy as np
2
3 DATASETS = ['Pantheon+', 'BAO_DESI', 'BBN']
4 ANALYSIS_NAME = "meu_wCDM"
5
6 PRIORS = {
7     'H0': (40.0, 90.0),
8     'Omega_cdm': (0.10, 0.50),
9     'Omega_b': (0.02, 0.06),
10    'w0': (-3.0, 1.0),
11    'M_B': (-21.0, -18.0), # nuisance para Pantheon+
12 }
13
14 _OMEGA_GAMMA_H2 = 2.469e-5 # densidade de fotons hoje
15
16 def H_model(z, p):
17     H0 = p['H0']
18     h = H0 / 100.0
19     Omega_m = p['Omega_cdm'] + p['Omega_b']
20     Omega_r = _OMEGA_GAMMA_H2 / h**2
21     w0 = p['w0']
22
23     Omega_DE = 1.0 - Omega_m - Omega_r # universo plano
24
25     H2 = (Omega_m * (1+z)**3
26           + Omega_r * (1+z)**4
27           + Omega_DE * (1+z)**(3*(1+w0)))
28
29     return H0 * np.sqrt(max(H2, 0.0))
```

Listing 2: theory_models/meu_wCDM.py

Para rodar:

```
1 python run.py --model meu_wCDM
```

O SERTAO detecta o novo arquivo automaticamente. Se o nome for digitado errado, uma lista dos modelos disponíveis é impressa na tela.

6.2 Diretrizes para a função H_{model}

- Retornar `np.nan` (não zero) quando $H^2 \leq 0$ — o engine interpreta `nan` como ponto fisicamente inválido e o rejeita corretamente.
- A função deve aceitar tanto escalares quanto arrays NumPy como argumento `z`. Use `np.where` para vetorizar guards: `H0 * np.sqrt(np.where(H2 > 0, H2, np.nan))`.

- Inclua sempre a componente de radiação ($\Omega_r = 2.469 \times 10^{-5}/h^2$) quando o dataset CMB `theta*` estiver ativo, pois ele é sensível à expansão em $z \sim 1090$.
- Para modelos de energia escura com $w \neq -1$: a componente de energia escura deve ser conservada separadamente (não absorvida em Ω_Λ).

7 Componentes Internos

Esta seção descreve os módulos internos do SERTA0. O usuário **não** precisa modificar nenhum deles para conduzir análises padrão.

7.1 `engine.py` — Motor de likelihood

O `engine.py` é o núcleo do framework. Para cada ponto θ proposto pelo sampler:

1. verifica o cache LRU — se o ponto já foi avaliado, retorna o resultado instantaneamente;
2. aplica a rejeição antecipada de Ω_m (garante $0 < \Omega_m < 1$ antes de qualquer cálculo);
3. constrói o objeto `GenericCosmology` com a função $H(z)$ do modelo;
4. pré-aquece o cache de distâncias até $z = 4$ em uma única operação, evitando reconstruções repetidas durante o cálculo das likelihoods;
5. avalia as likelihoods em ordem crescente de custo computacional (BBN $\rightarrow$ CMB $\rightarrow$ BAO $\rightarrow$ CC $\rightarrow$ RS Ia), interrompendo imediatamente ao encontrar $-\infty$.

A avaliação na ordem crescente de custo é importante: BBN e CMB são muito mais baratos que Pantheon+ (~ 1700 supernovas) e frequentemente rejeitam pontos inviáveis antes que o cálculo chegue aos datasets pesados.

Este arquivo não deve ser modificado pelo usuário.

7.2 `cosmology.py` — Distâncias e crescimento

Define a classe `GenericCosmology`, que recebe qualquer função $H(z)$ e fornece todas as grandezas cosmológicas necessárias para as likelihoods:

- **Distâncias:** comóvel D_C , luminosidade D_L , diâmetro angular D_A , BAO (D_M , D_H , D_V);
- **Horizonte de som:** fórmula de *fitting* r_d , com $N_{\text{eff}} = 3.044$ (Planck 2018);
- **Crescimento linear:** $D(z)$, $f(z)$, $\sigma_8(z)$, $f\sigma_8(z)$.

Cache de distâncias em dois segmentos. Para evitar a reconstrução repetida do cache durante o cálculo das likelihoods, o SERTA0 usa uma grade adaptativa:

- **Segmento 1** ($z < 4$, 1.000 pontos): cobre todos os datasets de baixo redshift (BAO, SNe, CC, RSD). Erro relativo $< 0.001\%$ — muito abaixo das incertezas observacionais de $\sim 0.5\%$.
- **Segmento 2** ($4 < z < z_{\text{CMB}}$, 5.000 pontos): construído apenas quando o dataset CMB θ_* está ativo. Erro relativo $< 0.007\%$ em $\chi(z^* \approx 1090)$.

A versão anterior usava 10.000 pontos em um único segmento de $z = 0$ até $z \approx 1150$. Isso causava dois problemas simultâneos: os pontos ficavam muito espaçados na região de baixo redshift (erro de 0.3% nos redshifts do BAO quando CMB estava ativo) e o cache era reconstruído múltiplas vezes por avaliação de likelihood. A estratégia de dois segmentos elimina ambos.

Equação de crescimento. A equação diferencial para o fator de crescimento linear é:

$$\frac{d^2 D}{da^2} + \left[\frac{3}{a} + \frac{1}{a} \frac{d \ln H}{d \ln a} \right] \frac{dD}{da} - \frac{3}{2} \frac{\Omega_m(a)}{a^2} D = 0, \quad (8)$$

onde $\Omega_m(a) = \Omega_{m,0}(1+z)^3 H_0^2 / H^2(z)$ e o termo $d \ln H / d \ln a = -\frac{3}{2} \Omega_m(a)$ é calculado analiticamente (válido para energia escura suave em RG), eliminando as três avaliações de $H(z)$ por passo da versão anterior.

A condição inicial é imposta em $z_{\text{ini}} = 50$ (matéria dominante), onde a solução atratora $D \propto a$ é exata a menos de 10^{-5} .

Este arquivo não deve ser modificado pelo usuário.

7.3 neutrinos.py — Neutrinos massivos

Implementa a evolução de $\Omega_\nu(z)$ usando o formalismo WMAP-7 (Komatsu et al. 2011). A função $f(y)$ interpola suavemente entre o regime relativístico [$f(0) = 1$, $\rho_\nu \propto a^{-4}$] e o regime não-relativístico [$f(y) \approx Ay$ para $y \gg 1$, $\rho_\nu \propto a^{-3}$].

A equação de estado efetiva $w_\nu(z)$ é derivada da derivada logarítmica de $\Omega_\nu(z)$, garantindo consistência interna. O redshift de transição para cada espécie i é $z_{\text{trans},i} \approx m_i / T_{\nu,0} - 1$. Para $m_i = 0.02 \text{ eV}$: $z_{\text{trans}} \approx 118$.

Este arquivo não deve ser modificado pelo usuário.

7.4 MCMC.py — Nested sampling

Implementa a amostragem via **dynesty** com as seguintes características:

- **Wrapper pickle-safe:** a função de likelihood é embrulhada em uma classe de nível de módulo (`_SERTAOLikelihood`), permitindo que o `dynesty` salve e restaure checkpoints corretamente via `pickle`;
- **Tabela ao vivo:** durante a execução, o terminal exibe uma tabela atualizada a cada 30 segundos com o estado atual da amostragem. O stdout real é capturado antes de qualquer redirecionamento, garantindo que as linhas apareçam mesmo com o output do `dynesty` suprimido;
- **MPI:** suporte nativo via `mpi4py`;
- **Saída GetDist:** as cadeias são salvas no formato padrão `weight | loglike | params....`

Este arquivo não deve ser modificado pelo usuário.

8 Função de Crescimento: Quando Modificar?

Para todos os modelos de energia escura suave em Relatividade Geral — Λ CDM, w CDM, CPL, quintessência — a equação de crescimento linear implementada é automaticamente correta. **Nenhuma modificação é necessária.** Basta fornecer a função $H(z)$ correta.

A modificação só é necessária quando o modelo introduz:

1. Gravidade modificada (ex.: $f(R)$, Horndeski, DGP);
2. Acoplamento direto matéria–energia escura;
3. Energia escura com clustering significativo;
4. Modificação explícita da equação de Poisson: $k^2\Phi = -4\pi G_{\text{eff}}(z) a^2 \rho_m \delta$.

Modelo	Modificar crescimento?
Λ CDM	Não
w CDM	Não
CPL ($w_0 w_a$ CDM)	Não
Quintessência	Não
$f(R)$, Horndeski	Sim
Acoplamento DE–DM	Sim

8.1 Extensão para gravidade modificada: $\mu(a)$

Em modelos de gravidade modificada, a equação de crescimento torna-se:

$$\frac{d^2 D}{da^2} + \left[\frac{3}{a} + \frac{1}{a} \frac{d \ln H}{d \ln a} \right] \frac{dD}{da} - \frac{3}{2} \frac{\mu(a) \Omega_m(a)}{a^2} D = 0, \quad (9)$$

onde $\mu(a) = G_{\text{eff}}(a)/G$. O caso padrão da RG é $\mu(a) = 1$.

Para implementar, adicione o parâmetro `mu_of_a` à classe `GenericCosmology` e modifique o termo fonte em `_growth_ode`:

```
1 # Em cosmology.py (dentro de _growth_ode):
2 mu = self.mu(a)    # mu(a) = 1 para Relatividade Geral
3
4 d2D_da2 = (
5     -(3.0/a + dlnH_dlnA/a) * dD_da
6     + 1.5 * mu * Omega_m_a * D / a**2
7 )
```

Parametrizações comuns de $\mu(a)$:

$$\mu(a) = 1 + \mu_0(1 - a) \quad (\text{linear em } a) \quad (10)$$

$$\mu(a) = 1 + \mu_0 \Omega_{\text{DE}}(a) \quad (\text{proporcional à fração DE}) \quad (11)$$

$$\mu(a) = 1 \quad (\text{GR padrão})$$

O caso $\mu_0 = 0$ deve recuperar exatamente o resultado da RG.

9 Módulo de Campo Escalar (`scalar_field.py`)

O módulo `scalar_field.py` implementa a dinâmica de quintessência canônica usando variáveis adimensionais no espaço de fase:

$$x \equiv \frac{\dot{\phi}}{\sqrt{6} H M_{\text{Pl}}}, \quad y \equiv \frac{\sqrt{V(\phi)}}{\sqrt{3} H M_{\text{Pl}}}, \quad \lambda \equiv -M_{\text{Pl}} \frac{V_{,\phi}}{V}. \quad (12)$$

A variável independente é o número de e -folds $N = \ln a$. As equações de evolução são:

$$\frac{dx}{dN} = -3x + \lambda \sqrt{\frac{3}{2}} y^2 + \frac{3}{2} x(1 + x^2 - y^2), \quad (13)$$

$$\frac{dy}{dN} = -\lambda \sqrt{\frac{3}{2}} xy + \frac{3}{2} y(1 + x^2 - y^2), \quad (14)$$

$$\frac{d\lambda}{dN} = -\sqrt{6} \lambda^2 (\Gamma - 1)x, \quad (15)$$

onde $\Gamma \equiv V V_{,\phi\phi} / (V_{,\phi})^2$ codifica a forma do potencial. Para adicionar um novo potencial, basta implementar $\Gamma(\lambda)$:

Potencial	Expressão	$\Gamma(\lambda)$
Exponencial	$V = V_0 e^{-\lambda\phi}$	$\Gamma = 1$
Lei de potência	$V = V_0 \phi^n$	$\Gamma = (n - 1)/n$
PNGB	$V = V_0[1 + \cos(\phi/f)]$	$\Gamma \approx 1 - 1/\lambda^2$

A partir das variáveis dinâmicas, obtemos:

$$\Omega_\phi = x^2 + y^2, \quad w_\phi = \frac{x^2 - y^2}{x^2 + y^2}. \quad (16)$$

A cosmologia resultante é passada ao `ScalarFieldCosmology`, que herda de `GenericCosmology` e delega todo o cálculo de distâncias e crescimento ao módulo `cosmology.py`.

10 Cosmologia Híbrida com CLASS (`hybrid_cosmology.py`)

O módulo `HybridCosmology` combina a infraestrutura interna do SERTA0 com o código Boltzmann CLASS para modelos que requerem grandezas perturbativas precisas:

- Espectro de potência da matéria $P(k, z)$;
- Função de transferência $T(k, z)$;
- $\sigma(R, z)$ e outras quantidades do CLASS.

A classe herda de `GenericCosmology` e mantém compatibilidade total com todas as likelihoods. Para usar, substitua `GenericCosmology` por `HybridCosmology` no modelo e configure o CLASS normalmente.

11 Testes Automatizados

O SERTA0 inclui uma suite de 21 testes funcionais que verificam o correto funcionamento de todas as likelihoods sem necessidade de CLASS ou de qualquer dependência externa além das já instaladas.

```
1 python test_likelihoods.py          # saída limpa
2 python test_likelihoods.py -v      # mostra valores de cada teste
```

Saída esperada:

```
1 =====
2 SERTA0 -- likelihood test suite
3 =====
4
5 PASS  BBN / fiducial logL
```

```

6 PASS BBN / parameter sensitivity
7 PASS BBN / peak at correct value
8 PASS BAO_DESI / fiducial logL
9 PASS BAO_DESI / sensitivity
10 ...
11 PASS Engine / BBN + BAO_DESI
12
13 =====
14 Results: 21 passed 0 failed 0 skipped / 21 total
15 =====

```

Cada teste verifica: (i) a likelihood retorna um valor finito e ≤ 0 para a cosmologia Planck 2018; (ii) o valor piora quando os parâmetros se afastam do melhor ajuste; (iii) a interface do engine integra corretamente todas as likelihoods.

Atenção

Recomenda-se executar `test_likelihoods.py` após qualquer modificação nos arquivos de `likelihoods/` ou em `SERTA0/cosmology.py` para garantir que nenhuma regressão foi introduzida.

12 Eficiência Computacional

12.1 Tempos esperados de execução

Os tempos abaixo foram medidos em uma máquina com processador moderno, análise single-core, `nlive = 300`:

Datasets	Custo/avaliação	Tempo total
BAO_DESI + BBN	~ 1 ms	~ 1 min
BAO_DESI + BBN + RSD	~ 3 ms	~ 4 min
BAO_DESI + BBN + Pantheon+	~ 16 ms	~ 20 min
BAO_DESI + BBN + CMB + Pantheon+	~ 17 ms	~ 22 min

12.2 Recomendações

- **Use MPI para publicações.** Quatro cadeias independentes garantem melhor exploração do espaço de parâmetros e permitem diagnósticos de convergência: `mpirun -np 4 python run.py -model LCDM`.
- **Aumente `nlive` para análises precisas.** O padrão `nlive=300` é adequado para exploração. Para resultados de publicação, use `nlive=500` ou mais: `python run.py -model LCDM -nlive 500`.

- **O cache é automático.** O engine mantém um cache LRU de até 50.000 avaliações. Pontos repetidos (frequentes no *rwalk* do *dynesty*) são retornados instantaneamente.
- **O pré-aquecimento do cache de distâncias é automático.** Após a construção do objeto cosmológico, o engine pré-aquece o cache de distâncias até $z = 4$ em uma única chamada. Sem isso, o cálculo das 7 distâncias do BAO DESI reconstruiria o cache 7 vezes por avaliação — um fator de lentidão de $\sim 7\times$ no custo de $H(z)$.

13 Resumo: O que o Usuário Precisa Fazer

1. **Escolher ou criar o modelo em `theory_models/`.** Para modelos padrão (LCDM, Λ CDM, w_0 CDM), apenas abrir o arquivo e ajustar DATASETS e os priors. Para um modelo novo, criar `theory_models/meu_modelo.py` com DATASETS, PRIORS e `H_model`.

2. **Executar a análise.**

```
1 python run.py --model meu_modelo
2 mpirun -np 4 python run.py --model meu_modelo    # com MPI
3
```

3. **Verificar o progresso na tela.** A tabela é atualizada a cada 30 segundos. Observar o valor de $\Delta\log Z$ decrescendo até < 0.1 .
4. **Analisar os resultados.** As cadeias em `chains/` estão no formato `GetDist` e prontas para uso com qualquer ferramenta compatível.

Nota

Nenhum arquivo de `SERTA0/` ou `likelihoods/` precisa ser modificado para análises padrão. Toda a configuração fica exclusivamente em `theory_models/`.